

Document Store Module

Case and project documents on MinIO object storage · Phase 1 implementation guide
Branch: `dev6` · 2026-09-28 · 613 tests green (125 new)

1. What Was Built

A document store where every document belongs to **exactly one owner** — a case (`Matter`) or a project (`Project`) — with bytes living in MinIO and only metadata in PostgreSQL. Two views over the same rows: a per-owner document panel and a firm-wide document library.

Area	Delivered in Phase 1
Storage	Generic MinIO service, bucket bootstrap, presigned POST uploads, presigned GET downloads
Metadata	<code>documents</code> table, <code>PENDING_UPLOAD</code> → <code>ACTIVE</code> → <code>ARCHIVED</code> lifecycle
Authorization	Permission gate + row-level scope for firm admins, assigned staff and clients
Quota	Per-firm allocation set by Super Admin, enforced under a row lock at confirm time
Integrity	MIME whitelist, extension whitelist, real size re-check, file-signature (magic byte) check
Sharing	Private by default; a client sees only documents explicitly shared with them
Audit	Upload / download / share / archive recorded against the document, plus case timeline entries
Deferred	Virus scanning, compression, per-firm KMS keys, hard delete & restore (see §11)

2. Architecture — Two Seams

The design separates *moving bytes* from *owning documents*, so the storage layer is reusable (invoice PDFs, scraper exports, report generation) and testable without a running MinIO.

2.1 Generic storage layer — `com.lawfirm.erp.common.storage`

<code>StorageService</code>	interface: <code>presignUpload</code> , <code>presignDownload</code> , <code>stat</code> , <code>readHead</code> , <code>delete</code> , <code>ensureBucket</code>
<code>MinioStorageService</code>	the ONLY class that imports <code>io.minio</code>
<code>StorageProperties</code>	<code>@ConfigurationProperties("storage.minio")</code> – every storage knob
<code>StorageConfig</code>	<code>MinioClient</code> bean, created only when an endpoint is configured
<code>StorageBootstrap</code>	creates the bucket at startup; logs instead of failing the boot
<code>StorageQuotaService</code>	per-firm allocation + usage counter
<code>FirmStorageUsage</code>	entity: one row per firm (<code>used_bytes</code> , <code>quota_bytes</code>)

Why an interface matters here. `DocumentService` depends on `StorageService`, never on MinIO. The whole test suite substitutes a 60-line in-memory double, so 125 tests run with no container, no network and no credentials.

2.2 Document module — `com.lawfirm.erp.modules.document`

controller/	<code>DocumentController</code>	firm staff API	<code>(/api/v1/firm/**)</code>
	<code>ClientDocumentController</code>	client portal	<code>(/api/v1/client/documents)</code>
service/	<code>DocumentService</code> / <code>DocumentServiceImpl</code> / <code>DocumentMapper</code>		
	<code>DocumentScopeGuard</code>	answers "may the caller see or act on this?"	
	<code>PendingUploadSweeper</code>	discards abandoned uploads	
entity/	<code>Document</code>		

repository/	DocumentRepository	three scoped queries
util/	DocumentStoragePath	readable, sanitised object keys
	DocumentTypePolicy	whitelist + file-signature rules
enums/	DocumentStatus, DocumentVisibility	

3. Data Model

3.1 documents

Column	Type	Purpose
<code>id</code>	BIGINT IDENTITY	Primary key (per requirement: int, not UUID)
<code>uuid</code>	UUID, unique	Stable identity for the audit trail — see below
<code>firm_id</code>	UUID, NOT NULL	Tenant key; every query filters on it
<code>matter_id</code>	UUID, NULL	Set when the document belongs to a case
<code>project_id</code>	UUID, NULL	Set when the document belongs to a project
<code>court_case_id</code>	UUID, NULL	Optional tag for one court instance of the matter
<code>original_filename</code>	VARCHAR(255)	As uploaded; never used to build the storage key
<code>content_type</code>	VARCHAR(100)	From the whitelist
<code>extension</code>	VARCHAR(10)	Derived from the content type, not the filename
<code>size_bytes</code>	BIGINT	Verified at confirm; quota counts this
<code>storage_key</code>	VARCHAR(500)	Object key. Authoritative pointer to the file
<code>etag</code>	VARCHAR(64)	From storage on confirm; normalised, never trusted raw
<code>visibility</code>	VARCHAR(10)	<code>PRIVATE</code> (default) or <code>SHARED</code>
<code>status</code>	VARCHAR(20)	<code>PENDING_UPLOAD</code> , <code>ACTIVE</code> , <code>ARCHIVED</code>
<code>uploaded_by_user_id</code>	UUID	Who confirmed the upload
<code>upload_expires_at</code>	TIMESTAMP	Upload window; drives the sweeper
<code>archived_at</code> , <code>archived_by_user_id</code>	—	Soft-delete trail
audit columns	—	<code>created_at/by</code> , <code>updated_at/by</code>

Indexes: `(firm_id, matter_id, status)`, `(firm_id, project_id, status)`, `(firm_id, created_at)`.

Why two nullable owner columns instead of `owner_type` + `owner_id`? Both sides are queried and indexed independently and need real foreign keys; a discriminator column cannot be joined or indexed per side. A database check constraint enforces the XOR (`documents_owner_xor_check`), and the service enforces it before insert.

Why the document also carries a `uuid`. `audit_logs.entity_id` is a `UUID` column, and `AuditAspect` only accepts a `UUID` or a `UUID-parsable string`. A numeric primary key therefore produced `entityId = null` — the audit row would have existed but pointed at nothing. The document's `uuid` is what the audit trail records.

3.2 firm_storage_usage

Column	Type	Purpose
id	BIGINT IDENTITY	Primary key
firm_id	UUID, unique, NOT NULL	One row per firm
used_bytes	BIGINT	Bytes currently counted against the firm
quota_bytes	BIGINT	Allocated bytes. 0 means unlimited

Why quota lives here and not in `firm_configs` . `FirmConfigService` is writable by a Firm Admin through the firm settings API. If the allocation were a firm config key, a Firm Admin could raise their own limit. Keeping it in a table with a Super-Admin-only endpoint makes allocation a platform decision, and puts the counter and the limit in the same row so one lock covers both.

4. Storage Layout

```
firms/{firmId}/cases/{matterNumber}/{objectId}/{filename}
firms/{firmId}/projects/{projectCode}/{objectId}/{filename}

firms/8c2f1a3e-.../cases/MT-2026-00041/b606d326-.../petition.pdf
firms/8c2f1a3e-.../projects/ABC-PRJ-2026-00007/f9aba4b5-.../agreement.pdf
```

- **Case number / project code in the path** — what a human browsing the MinIO console expects to see. Both are firm-unique and effectively immutable (`projectCode` is declared `updatable = false`).
- **A UUID folder per document** — filenames can collide, so the readable part never has to be uniquified. The id is generated at ticket time, because the row needs its key before insert.
- **firmId first, not a firm code** — tier one must be immutable; a firm code can change and would orphan every object beneath it.
- **Filename sanitised** — path separators, control characters and leading dots removed, length capped at 120 keeping the extension.
- **storage_key is authoritative.** If a case number were ever changed, existing rows still point at the real object; the readable path is a convenience, not a lookup key.
- **Uploads cannot escape the firm prefix.** The POST policy pins an exact key, and every generated key starts with `firms/{firmId}/` .

S3 has no real folders — slashes in the key are simply what MinIO renders as a tree.

5. Upload Flow

Bytes never pass through the application. The client uploads straight to MinIO against a presigned POST policy, so a 50 MB file costs the API process nothing.

```
POST /api/v1/firm/documents/upload-ticket
→ validate owner (exactly one of matterNumber / projectCode) and the caller's access
→ validate filename extension + content type against the whitelist
→ validate declared size against the 50 MB cap
→ advisory quota check (fail fast, before any bytes move)
→ insert Document (status = PENDING_UPLOAD, upload_expires_at = now + 30 min)
→ return { documentId, uploadUrl, fields{key, Content-Type, policy, x-amz-signature}, expiresAt }
```

Client POSTs `fields` + the file, as multipart/form-data, straight to MinIO

```
POST /api/v1/firm/documents/{id}/confirm
→ status must still be PENDING_UPLOAD and within the window
→ stat(key): object must exist, real size must be <= the cap and EQUAL the declared size
→ readHead(key, 4096): file signature must match the declared content type
→ quotaService.reserve(firmId, size) ← row lock, or the object is deleted and the call fails
```

→ status = ACTIVE, uploaded_by = caller, etag stored
→ case documents append a matter timeline event

5.1 Why presigned POST rather than presigned PUT

A presigned *PUT* URL is a bearer credential for one key and cannot limit the size written to it — MinIO would accept a single PUT up to 5 GB. The 50 MB cap and the quota check would then be advisory, only discovered after the bytes had landed. A POST *policy* carries explicit conditions that storage itself enforces:

Policy condition	Effect
content-length-range 1 → 50 MB	An oversized upload is refused before it is stored
eq on key	The object can only be written to the exact key issued
eq on Content-Type	A file cannot be stored under a different declared type

Every condition is inside the signature, so a tampered field is rejected by storage rather than silently accepted.

5.2 Defence in depth on the file itself

A declared `Content-Type` is attacker-controlled, so it is only the first gate. `confirm` re-reads the object and checks its leading bytes:

Declared type	Required signature
application/pdf	%PDF
docx / xlsx / pptx / zip	PK (ZIP container)
image/jpeg / image/png	FF D8 FF / 89 50 4E 47
image/tiff	II*\0 or MM*\0*
doc / xls / ppt	OLE2 header D0 CF 11 E0 A1 B1 1A E1
text/plain , text/csv	No signature — rejected if a NUL byte appears in the head

An executable renamed to `petition.pdf` with a spoofed header fails here, and the object is deleted rather than left in the bucket.

5.3 One whitelist, not two

The store originally consulted two different lists: `DocumentTypePolicy.ALLOWED_TYPES` for the content type, and the application-wide `AllowedExtensions` enum for the filename. They did not agree.

Case	Old behaviour
scan.tiff , bundle.zip	Content type is whitelisted, but the extension gate refused them — so two supported types were unreachable
animation.gif , logo.svg	Passed the extension gate, then failed the content-type gate with an unrelated-sounding message

`DocumentTypePolicy` now derives its accepted extensions *from* `ALLOWED_TYPES` (`allowedExtensions()` , plus the `jpeg` alias), so the two gates cannot drift apart again and the error message lists exactly the set that works:

`pdf doc docx xls xlsx ppt pptx txt csv jpg jpeg png tiff zip` . Three tests pin the agreement in both directions, including that an extension with no allowed content type is refused at the first gate.

5.4 Quota enforcement and the compensation path

The authoritative check happens at confirm, inside one transaction:

```
lock the firm's usage row (SELECT ... FOR UPDATE)
  quota = 0           → unlimited, never refused
  used + size > quota → delete the object, throw a readable error
  otherwise           → used_bytes += size
```

The compensating delete is not optional. By the time confirm runs, the bytes are already in the bucket. Refusing the upload without deleting the object would leak storage forever and leave an orphan nothing points at. There are three such paths — oversized, size mismatch, signature mismatch — plus the quota path, and each one deletes the object before failing.

The lock is what stops two concurrent confirms from both fitting into the last of the allocation. The pre-flight check at ticket time reads the counter without the lock, purely so an over-quota upload is refused before the client bothers sending anything.

6. Authorization

Three layers, all of which must pass.

Layer	Mechanism	What it decides
1	<code>JwtAuthFilter</code>	Valid token, active user, firm not suspended — already enforced per request
2	<code>PermissionEvaluator.require(...)</code>	Which <i>actions</i> the caller's role may take
3	<code>DocumentScopeGuard</code> + repository queries	Which <i>documents</i> those actions apply to

6.1 Who sees what

Caller	Documents reachable
Firm admin (<code>FIRM</code>)	Every document in the firm. Not narrowed by assignment
Super admin	Platform-wide; allocates space per firm
Firm staff (<code>FIRM_USER</code>)	Only cases in <code>case_assignments</code> and projects in <code>project_members</code> — <i>and</i> only the actions the firm admin granted their role
Client (<code>CLIENT</code>)	Only their own case/project, only <code>SHARED</code> , only <code>ACTIVE</code> . Never uploads or changes anything

"The firm admin must give permission" is satisfied by the existing role/permission system the firm admin already manages in Role Management — no second, per-document grant table was introduced. The three permissions are `DOCUMENT_MANAGEMENT:VIEW` (view + download), `:UPLOAD` (upload + confirm) and `:EDIT` (archive), plus `:SHARE` for making a document client-visible.

6.2 How the scope is expressed in SQL

Three separate queries, one per scope. The caller's scope selects exactly one of them, so no request can widen its own result set by changing a parameter. Scope is expressed as subqueries, which means an employee with no assignments naturally gets an empty page rather than needing an empty- `IN` guard.

```
Firm admin  findForFirm(firmId, filters)

Firm staff  findAssignedTo(firmId, userId, filters)
            ... AND ( d.matterId IN (SELECT ca.matterId FROM CaseAssignment ca WHERE ca.userId = :userId)
            OR d.projectId IN (SELECT pm.projectId FROM ProjectMember pm WHERE pm.userId = :userId) )

Client      findVisibleToClient(firmId, userId, SHARED, ACTIVE, filters)
```

```
... AND d.visibility = 'SHARED' AND d.status = 'ACTIVE'
... AND ( d.matterId IN (SELECT m.id FROM Matter m WHERE m.clientUserId = :userId)
      OR d.matterId IN (SELECT mp.matterId FROM MatterParty mp
                        WHERE mp.clientId = :userId AND mp.isOurClient = true)
      OR d.projectId IN (SELECT p.id FROM Project p WHERE p.clientUserId = :userId) )
```

The client query deliberately ignores any `visibility` or `status` filter the caller supplies, and the party branch keeps matters created before the `clientUserId` column existed visible to their client.

Cross-firm requests return 404, not 403. Every document lookup goes through `findByIdAndFirmId(id, callerFirmId)`, so another firm's row simply does not exist as far as the caller is concerned. This avoids confirming that a guessed id is real.

7. Sharing With Clients

Uploads start `PRIVATE` — no document reaches a client by accident, which matters when the same case holds internal notes, fee memos and strategy drafts. A user holding `DOCUMENT_MANAGEMENT:SHARE` promotes a document to `SHARED`:

```
PATCH /api/v1/firm/documents/{id}/visibility { "data": { "visibility": "SHARED" } }
```

Each change is audited. Archived documents cannot be shared, and a client's effective access is the intersection of *shared* and *their own case* — either alone is not enough.

8. Per-Firm Storage Allocation

The platform decides how much space each firm gets: 5 GB for firm A, 10 GB for firm B.

Endpoint	Actor
PUT /api/v1/super-admin/firms/{firmId}/storage-quota	Super Admin (@PreAuthorize("hasRole('SUPER_ADMIN')"))
GET /api/v1/super-admin/firms/{firmId}/storage-usage	Super Admin
GET /api/v1/firm/documents/storage-usage	Any firm user with <code>:VIEW</code> — own firm only

```
PUT .../storage-quota { "data": { "quotaBytes": 10737418240 } } // 10 GB

{
  "usedBytes": 3221225472, "quotaBytes": 10737418240,
  "availableBytes": 7516192768, "usedPercent": 30.0, "unlimited": false
}
```

- `quotaBytes: 0` means unlimited — the default for a firm with no explicit allocation is `DEFAULT_STORAGE_QUOTA_BYTES` (5 GB).
- A newly created firm gets its row lazily on first upload; firms that predate the feature are backfilled at startup by `StorageUsageBootstrap`.
- Lowering an allocation never deletes existing documents — it only blocks new uploads.
- Archiving a document releases its bytes immediately.
- An Employee with a role lacking these permissions cannot read the firm's usage.

9. Lifecycle, Audit and Timeline

9.1 Status transitions

ticket issued → PENDING_UPLOAD
confirm ok → ACTIVE (counts against quota)
DELETE → ARCHIVED (bytes released; file KEPT)
window lapsed → row discarded by the sweeper (never became a document)

Delete is always a soft delete. The stored object is never removed: legal documents are kept for retention. Hard deletion after a retention period is deliberately a separate, explicitly authorised operation and is not part of Phase 1.

9.2 Audit entries

Action	Audit action	Recorded when
Upload confirmed	DOCUMENT_UPLOADED	A file passes every check and becomes ACTIVE
Download link issued	DOCUMENT_DOWNLOADED	Every presigned download URL
Visibility changed	DOCUMENT_SHARED	Private ⇌ shared, including who did it
Archived	DOCUMENT_DELETED	Soft delete

Each row carries the firm, the user, the document's `uuid` as `entity_id`, and a human-readable summary. **Presigned URLs are never logged** — the audit summary contains only the filename, because the URL is itself a credential. A test asserts that no audit summary ever contains `http`.

Two pre-existing bugs found and fixed while wiring this up. The `@Audit` annotation mechanism had never worked at all:

- `AuditAspect` resolved the caller only from the `SecurityContext` principal, but the principal is an `AuthenticatedDetail`; the `AuthenticatedUser` is published as a *request attribute* (which is how `AuditServiceImpl` reads it). The aspect therefore returned early for every request.
- `AuditSpelHelper` set the result as the SpEL *root object* but never bound `#result` as a variable, so every documented `#result.x` expression threw and evaluated to null.

Together these meant no annotated method ever wrote an audit row. Both are fixed centrally, so every existing `@Audit` endpoint across the application now records properly.

9.3 Case timeline

A confirmed document on a case appends a `MatterTimelineEvent` (`DOCUMENT_UPLOADED`, title "Document uploaded: petition.pdf", tagged with the court instance when one was given), so uploads appear alongside hearings and judgments in the case feed. Projects have no timeline entity, so project documents do not create one.

10. API Reference

10.1 Firm staff — `/api/v1/firm`

Endpoint	Permission	Notes
POST <code>/documents/upload-ticket</code>	:UPLOAD	Body: matterNumber or projectCode, filename, contentType, sizeBytes, optional courtCaseRef
POST <code>/documents/{id}/confirm</code>	:UPLOAD	Body optional

GET /documents	:VIEW	Library. Filters: status, visibility, search, page, size
GET /matters/{matterNumber}/documents	:VIEW	Case panel
GET /projects/{projectCode}/documents	:VIEW	Project panel
GET /documents/{id}/download-url	:VIEW	15-minute presigned URL
PATCH /documents/{id}/visibility	:SHARE	PRIVATE ⇌ SHARED
DELETE /documents/{id}	:EDIT	Archive (soft)
GET /documents/storage-usage	:VIEW	Own firm only

10.2 Client portal — /api/v1/client/documents

Endpoint	Notes
GET /	Own case/project, shared and active only. Optional matterNumber / projectCode / search
GET /{id}/download-url	Shared documents on the caller's own case only

Download is gated on :VIEW , not a separate :DOWNLOAD . In this product "view" means access to the document, and the seeded matrix gives READ_ONLY roles (paralegal) and clients only ACCESS + VIEW . Gating downloads on :DOWNLOAD would have locked paralegals and clients out of reading any file at all — the same shape as QA finding F-14.

11. Configuration

```
# application.yml
storage:
  minio:
    endpoint: ${MINIO_ENDPOINT:http://localhost:9000}
    access-key: ${MINIO_ACCESS_KEY:minioadmin}
    secret-key: ${MINIO_SECRET_KEY:minioadmin}
    bucket: ${MINIO_BUCKET:tarikh-documents}
    region: ${MINIO_REGION:us-east-1}
    upload-expiry-seconds: 1800    # 30 minutes
    download-expiry-seconds: 900  # 15 minutes
    max-file-size-bytes: 52428800 # 50 MB
    default-quota-bytes: 5368709120 # 5 GB for a firm with no explicit allocation
```

MINIO_ENDPOINT must be reachable from the browser. Presigned URLs are generated against it and the browser fetches the file directly. An internal-only hostname (http://minio:9000) works from the API container but breaks every upload at runtime. Storage also needs a CORS rule allowing the frontend origin's POST — a missing rule surfaces as an opaque browser error, not an auth error.

Local MinIO: docker compose up -d (API on localhost:9000 , console on localhost:9001). The bucket is created automatically at startup and stays private.

11.1 Database migration

Hibernate ddl-auto=update creates the two new tables, but it does not create check constraints. Run once per environment:

```
src/main/resources/db/migration/V2026_09_28__document_store_constraints.sql
```

It adds `documents_owner_xor_check` and rebuilds the enum check on `matter_timeline.event_type` to include the new `DOCUMENT_UPLOADED` value — the same trap that needed a migration when `permissions.action` gained new values.

11.2 Dependencies

```
io.minio:minio:8.6.0
com.squareup.okhttp3:okhttp-jvm:5.1.0 ← required, see note
```

Why `okhttp-jvm` is declared explicitly. MinIO 8.6.0 depends on `okhttp` 5.x, but the `okhttp` artifact at 5.x is a Kotlin-multiplatform metadata jar containing *no classes* — the JVM classes live in `okhttp-jvm`. Gradle resolves that variant from module metadata; Maven does not, so without this line compilation fails on `okhttp3.HttpUrl`. MinIO 8.x (the pre-9.0 line) is pinned deliberately: 9.0.0 was a full rewrite of the SDK.

12. Testing

Suite	Tests	Focus
<code>DocumentStoragePathTest</code>	9	Key layout, path-traversal, sanitising, long names
<code>DocumentTypePolicyTest</code>	45	Whitelist, extensions, every file signature
<code>StorageQuotaServiceTest</code>	14	Quota arithmetic, unlimited mode, counter floors, allocation
<code>DocumentServiceImplTest</code>	36	Flows and authorization with all collaborators mocked
<code>DocumentQaTest</code>	21	End-to-end over real HTTP + real security

Full suite: 613 tests, 0 failures, 0 errors.

12.1 How the E2E suite drives a real upload

An in-memory `StorageService` replaces MinIO. The test calls the real ticket endpoint, reads the generated key out of the response, "uploads" genuine file bytes under that key, then calls confirm — so the size check, the magic-byte check, the quota reservation and the audit write all run for real:

```
JsonNode ticket = POST /firm/documents/upload-ticket { matterNumber, filename, contentType, sizeBytes }
storageService.put(ticket.fields.key, DocumentTestFiles.pdf("...")) // the browser's POST
POST /firm/documents/{documentId}/confirm → 200 ACTIVE
```

Documents with real signatures (`%PDF` , `PK` , `FF D8 FF` , `89 50 4E 47`) are used, and the disguised-executable test uploads a genuine `MZ` header under a PDF label to prove the signature check rejects it.

12.2 Authorization covered

- Firm admin sees every document in the firm, case and project alike
- Assigned advocate sees only their case's document; an unassigned advocate sees an empty library
- A project member sees and may add to their project; a non-member may not upload to it
- Client cannot see an unshared document, then can after sharing; cannot upload, share or archive
- Client cannot reach another client's case even when a document on it is shared
- Another firm's admin sees an empty library and gets 404 on ids that exist in firm A
- A role with `:VIEW` only can read but not upload; a role with no document permission cannot list
- A firm admin cannot raise its own storage allocation

- Audit rows exist for all four actions and never contain a URL

13. Known Limitations & Next Phases

Not in Phase 1	Note
Virus scanning	<code>PENDING_SCAN</code> / <code>QUARANTINED</code> statuses were deliberately left out rather than sitting dead in the enum. The <code>StorageService</code> seam is where a scan step drops in
Compression	Tested expectation: gzip barely helps docx/xlsx (already ZIP) or images, and only text benefits. Sizes are stored as one <code>size_bytes</code> ; a split original/stored pair comes with the feature
Per-firm KMS keys	Bucket-level encryption only. Per-firm keys need KES and a key per <code>firm_storage_config</code> row
Hard delete / restore	Archiving never destroys the file. Un-archiving and retention-based purging are separate operations needing their own authorization and audit
Multipart upload	Single POST up to 50 MB. Larger files need a multipart flow with part-level presigning
Bulk upload & export	No ZIP-a-case or bulk-upload endpoint yet; <code>:DOWNLOAD</code> remains reserved for that

13.1 Operational notes

- **Backups are now two systems.** Postgres holds metadata, MinIO holds every file. A database-only restore yields rows whose `storage_key` points at missing objects.
- The bucket must stay private. Nothing should ever be served without a presigned URL.
- Pre-signed URL lifetimes are configuration, not code: shorten them if a firm's threat model demands it.
- The sweeper runs every 15 minutes and discards `PENDING_UPLOAD` rows past their 30-minute window, removing any partial object.

13.2 Verification commands

```
./mvnw -o test                                # full suite
./mvnw -o test -Dtest=DocumentQaTest           # end-to-end authorization suite
./mvnw -o test -Dtest='Document*Test,StorageQuotaServiceTest'
```

Run the migration SQL against a Postgres environment before the first boot on it, then confirm:

```
SELECT COUNT(*) FROM documents WHERE (matter_id IS NOT NULL) = (project_id IS NOT NULL); -- must be 0
```

Finally, verify against real MinIO that your server version *rejects an oversized POST* (a 2016 bug report describes old releases ignoring `content-length-range`). If it does not, a `stat` -based size backstop is needed.